

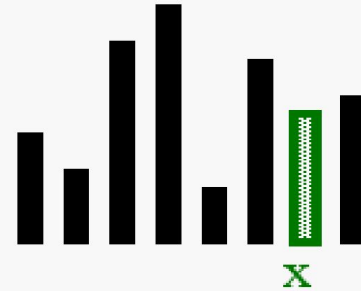
Lecture 09

Md. Nazrul Islam Mondal

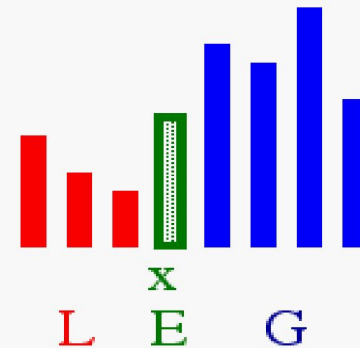
Quick Sort

Idea of Quick Sort

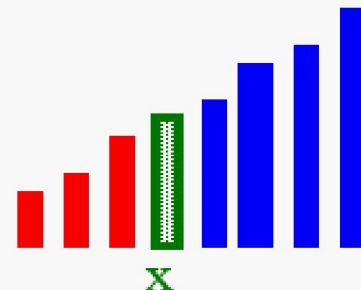
1) **Select:** pick an element



2) **Divide:** rearrange elements so that **x** goes to its **final position E**

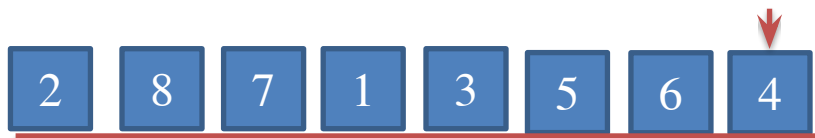
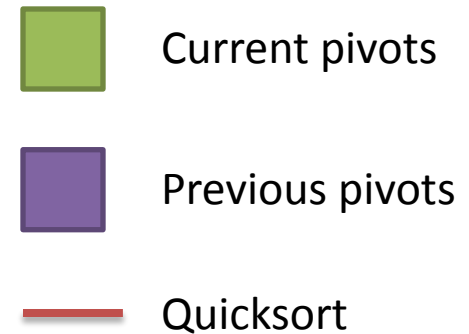


3) **Recurse and Conquer:** recursively sort



Quick Sort: Algorithm

- Quicksort example



Hi, I am nothing

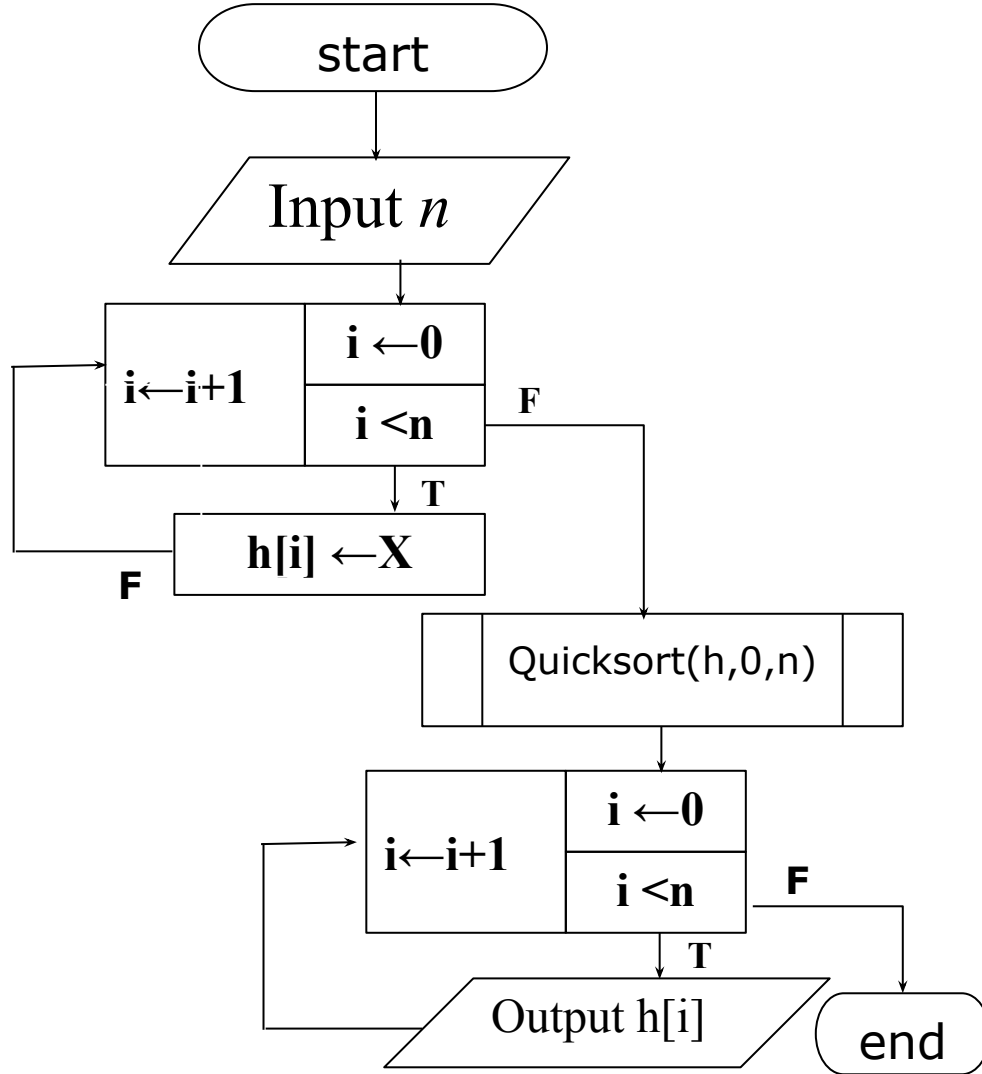


Nothing Jr.

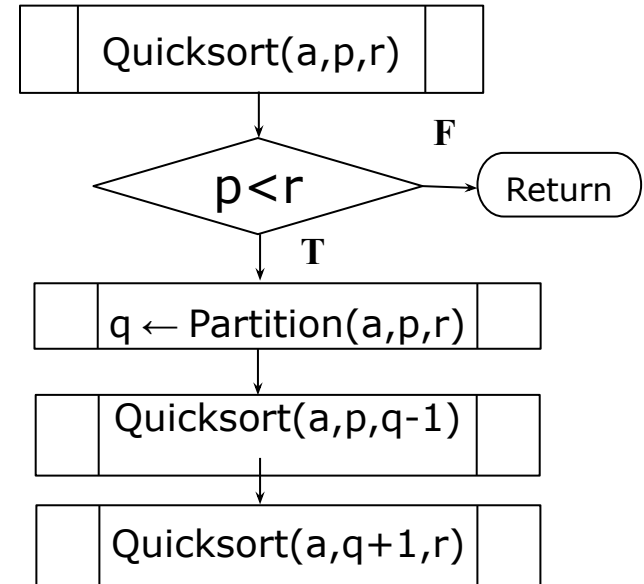
Nothing 3rd



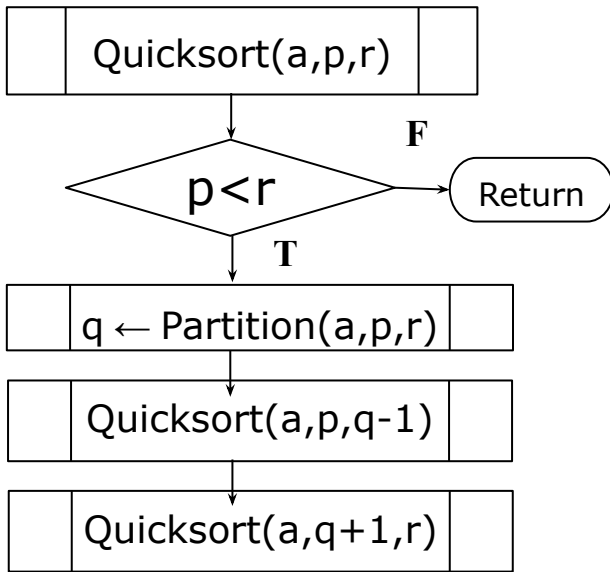
Quick Sort: Algorithm



Flowchart



Quick Sort: Algorithm

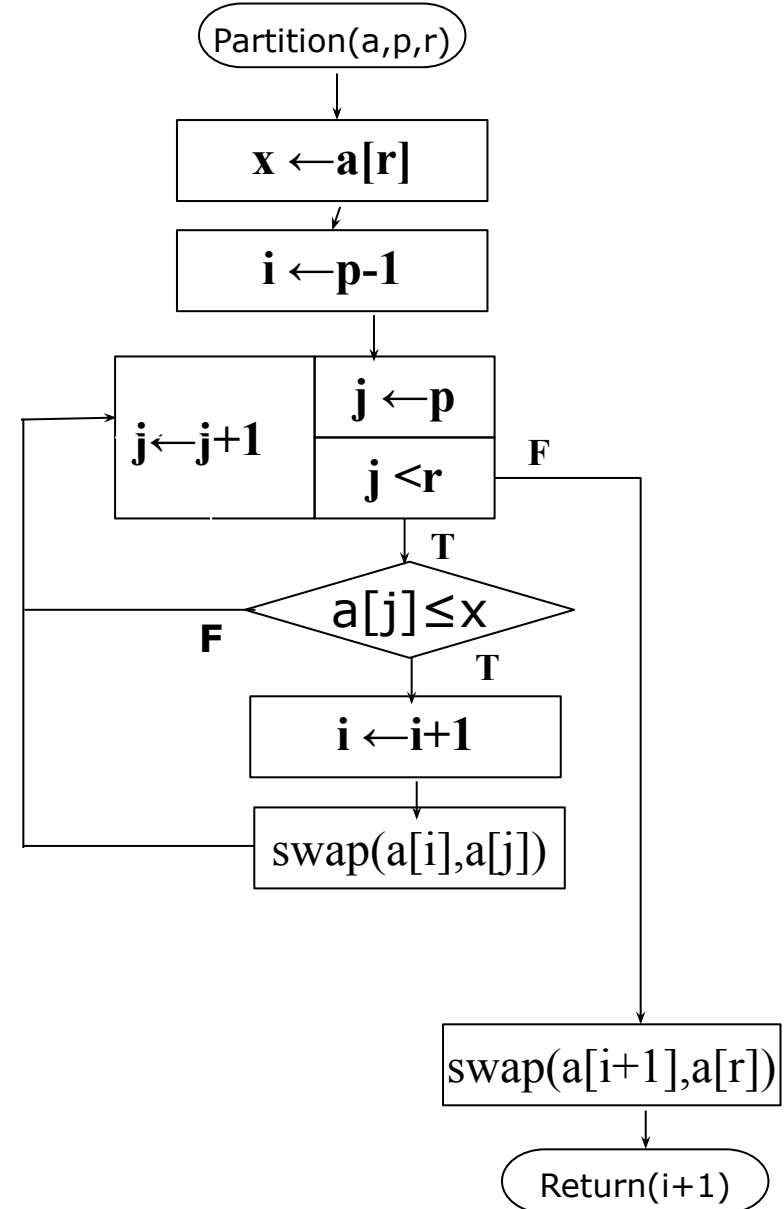


$a[6] = \{5, 7, 6, 1, 3, 2, 4\}$

i : puts elements $< x$ at left

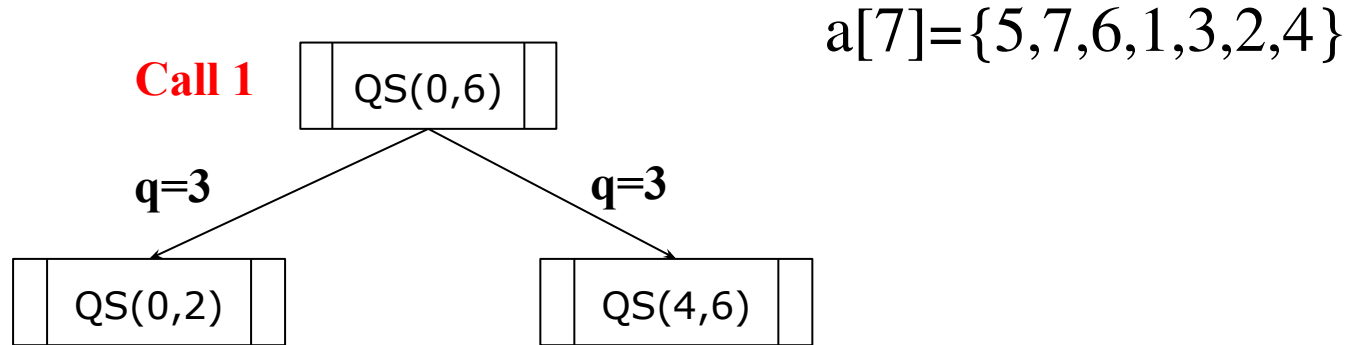
j : search all element except x

Flowchart



Quick Sort: Algorithm

Function Calling



Range

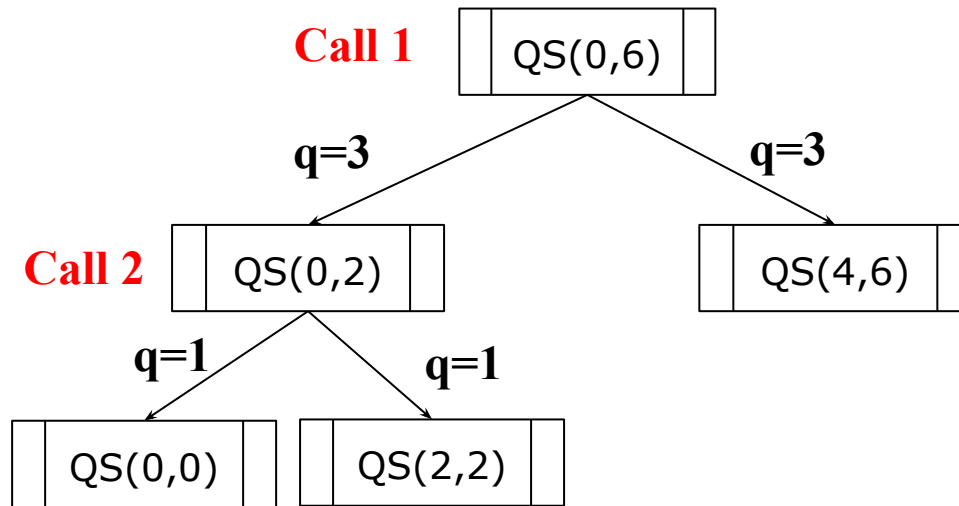
Left: $p - (q-1)$

Right: $(q+1) - r$

Quick Sort: Algorithm

Function Calling

$a[7] = \{5, 7, 6, 1, 3, 2, 4\}$



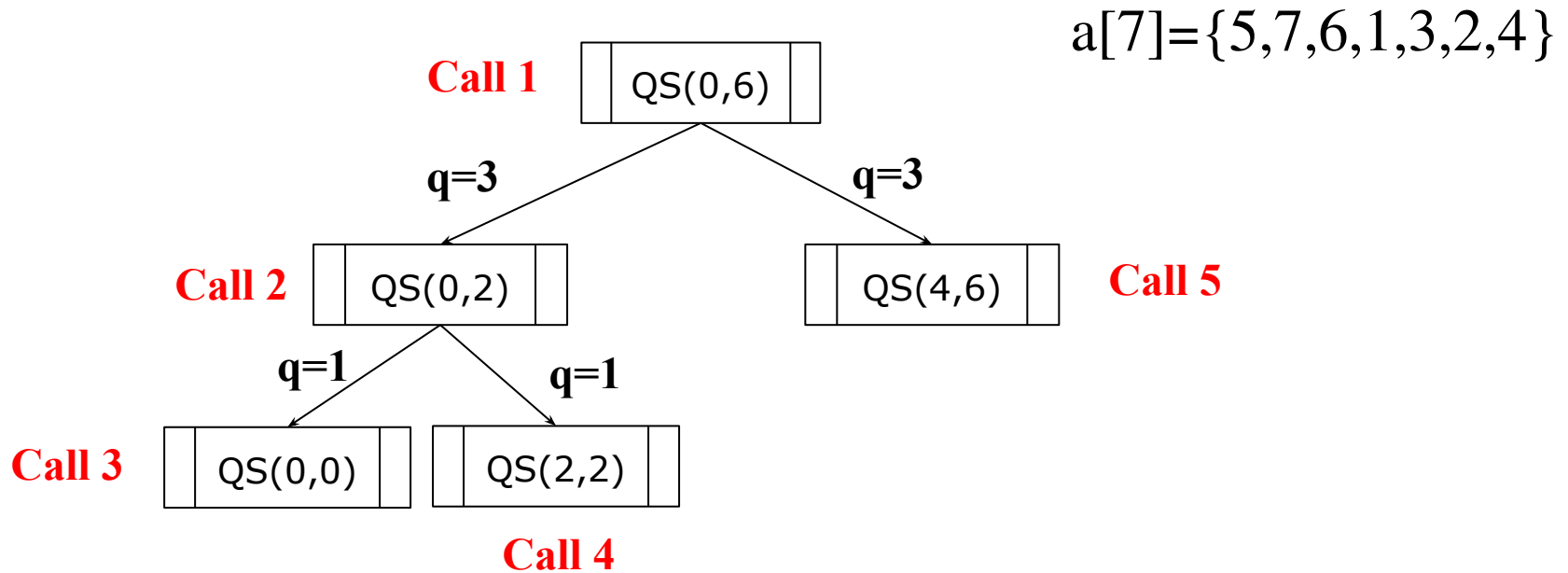
Range

Left: $p - (q-1)$

Right: $(q+1) - r$

Quick Sort: Algorithm

Function Calling



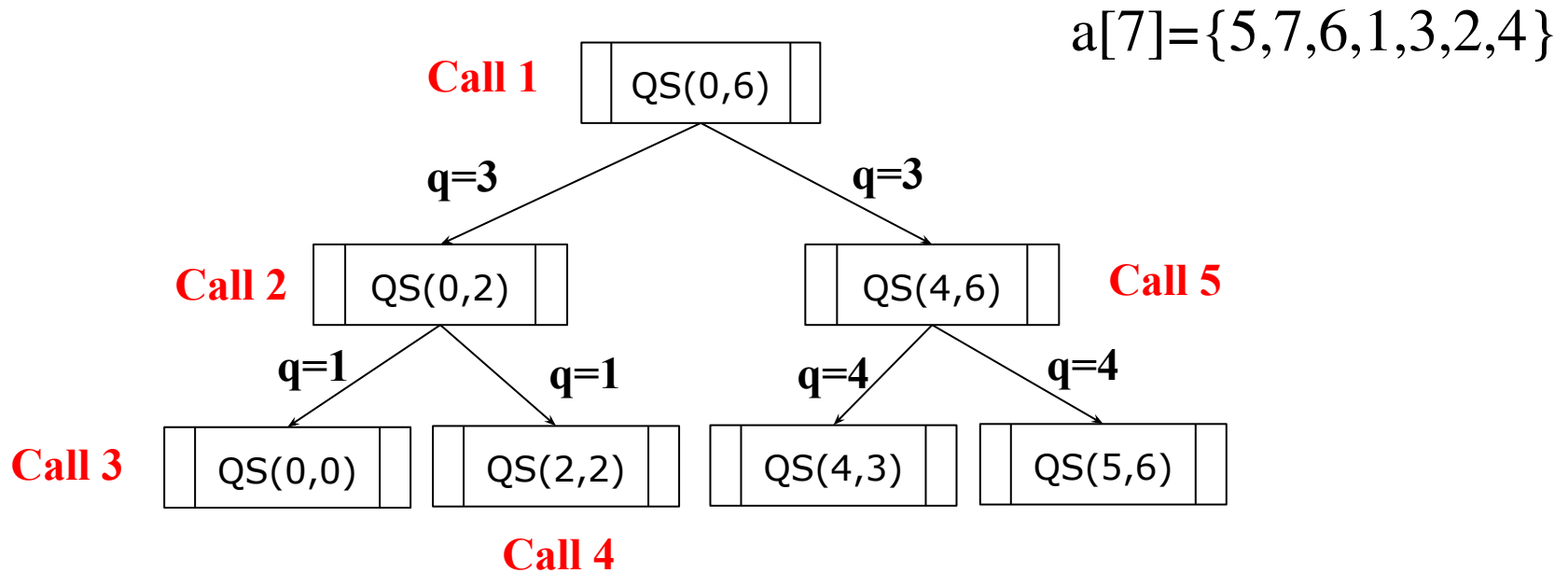
Range

Left: $p - (q-1)$

Right: $(q+1) - r$

Quick Sort: Algorithm

Function Calling



Range

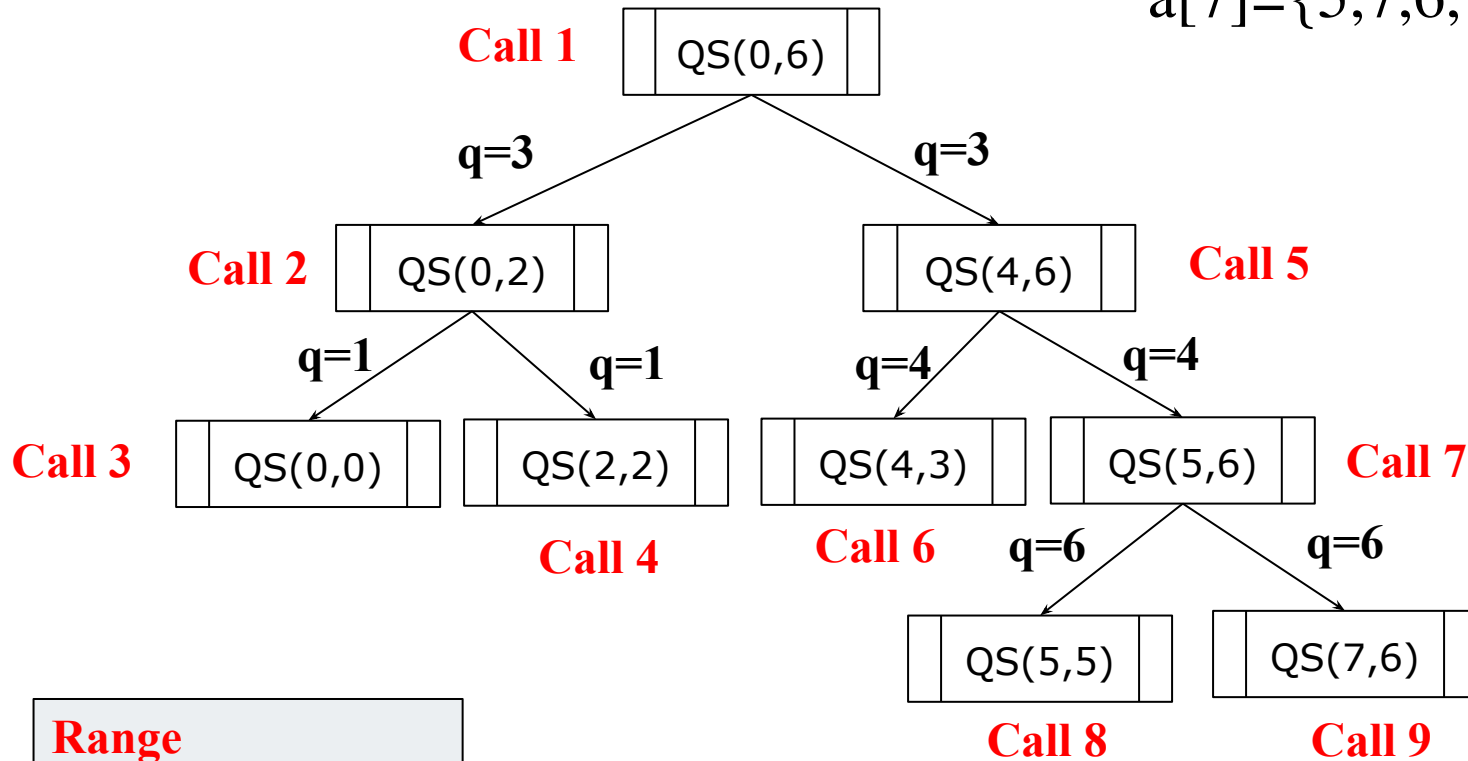
Left: $p - (q-1)$

Right: $(q+1) - r$

Quick Sort: Algorithm

Function Calling

$a[7] = \{5, 7, 6, 1, 3, 2, 4\}$



Range

Left: $p - (q-1)$

Right: $(q+1) - r$

Quick Sort: Algorithm

```
#include <stdio.h>
#include <stdlib.h>
void QuickSort(int *a,int p,int r);
int Partition(int *a,int p,int r);
int main()
{
    int a[7]={5,7,6,1,3,2,4};
    int p,r,i;
    p=0;r=6;
    QuickSort(a,p,r);
    for(i=p;i<=r;i++)
        printf("%d ",a[i]);
    return 0;
}
```

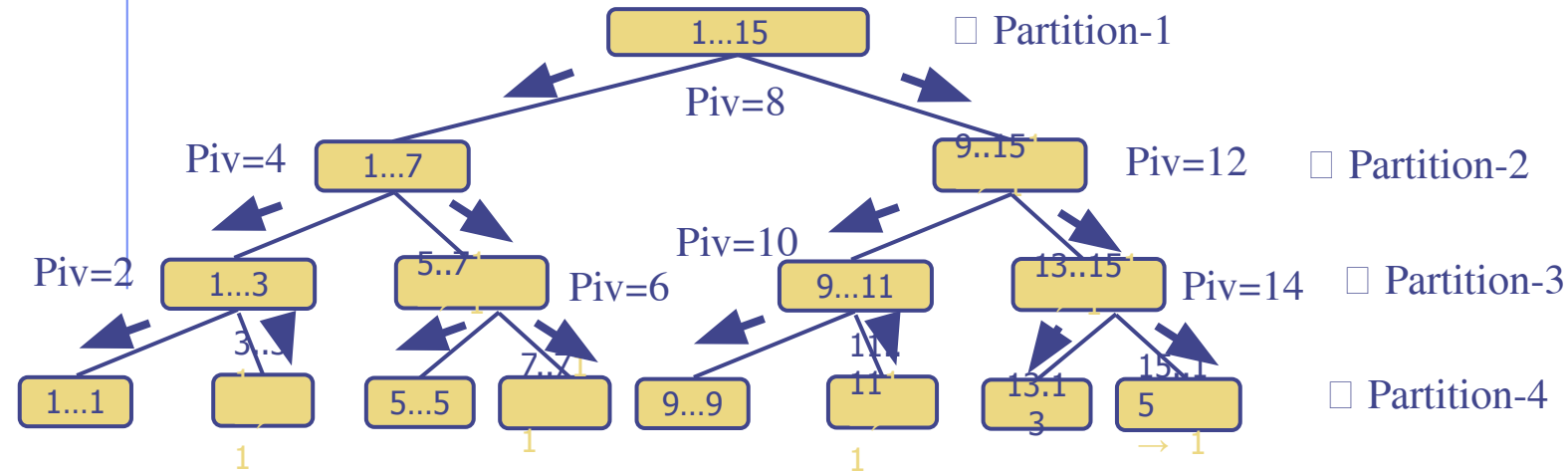
```
void QuickSort(int *a,int p,int r){
    int i,q;
    //printf("p=%d r=%d\n",p,r);
    if(p<r){
        q=Partition(a,p,r);
        printf("q=%d\n",q);
        QuickSort(a,p,q-1);
        QuickSort(a,q+1,r);
        //printf("q=%d",q);
    }
}
```

```
int Partition(int *a,int p,int r){
    int x,j,t,i;
    x=a[r]; i=p-1;
    for(j=p;j<=r-1;j++){
        if(a[j]<=x){
            i++;
            t=a[i];a[i]=a[j];a[j]=t;
        }
    }
    t=a[i+1];a[i+1]=a[r];a[r]=t;
    return(i+1);
}
```

Quick Sort: Algorithm

Expected Running Time

- Consider an array with 15 elements
 - Best Case:** Pivot always places at the middle of the list



No of elements in each sub-list after 1st Partition= $n/2$

No of elements in each sub-list after 2nd Partition= $n/2/2$

No of elements in each sub-list after kth Partition= $n/2^k$

Finally, there will be only one element per sub-list

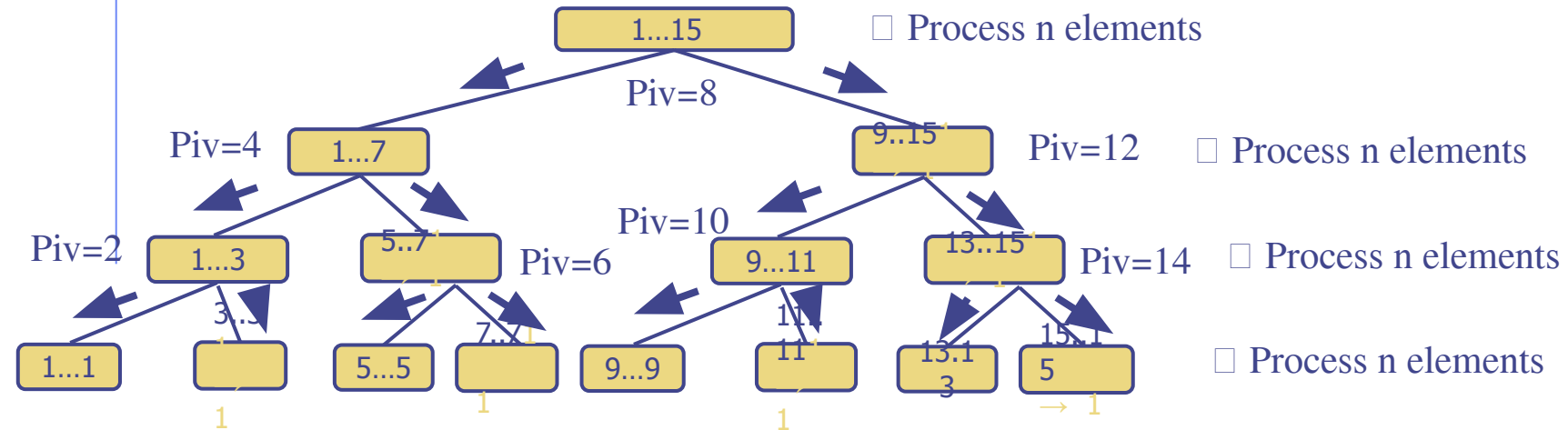
$$\square \quad n/2^k=1$$

$$\square \quad k=\log_2 n$$

Quick Sort: Algorithm

Expected Running Time

- Consider an array with 15 elements
 - Good call:** Pivot always places at the middle of the list



It is a binary tree with level k

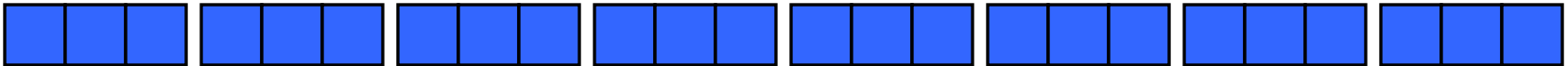
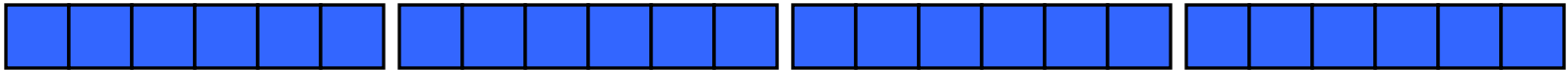
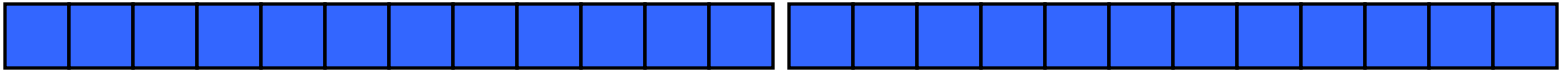
□ $k = \log_2 n$

In each level n elements are processed
So Time Complexity = $O(n \log_2 n)$

Select median as pivot
Practically impossible

Quick Sort: Algorithm

Partitioning at various levels



Quick Sort: Algorithm

Worst-case Running Time

- The worst case for quick-sort occurs when the pivot is the unique minimum or maximum element
- One of L and G has size $n - 1$ and the other has size 0
- The running time is proportional to the sum

$$n + (n - 1) + \dots + 2 + 1$$

- Thus, the worst-case running time of quick-sort is $O(n^2)$

depth time

